Assignment 5: A Class-Based Ride Sharing System in Smalltalk and C++

Rahul Solanki

University of the Cumberlands

MSCS-632-M30: Advanced Programming Language

Jay Thom

November 8, 2026

Assignment 5: A Class-Based Ride Sharing System in Smalltalk and C++

I built the same ride sharing system in GNU Smalltalk 3.2.5 and in C++17. Each version has a `Ride` base class with three subclasses (`StandardRide`, `PremiumRide`, and `SharedRide`, which extends `StandardRide`), a `Driver` class, a `Rider` class, and a `RideSystem` class that gives each requested ride to the driver with the fewest rides. Premium rides go only to drivers rated 4.8 or higher. Both programs print the same 56 lines after their first line (Figure A14). The code is at https://github.com/iknalos/MSCS-632-M30-Assign-5, and the Appendix shows the code and sample runs.

Encapsulation

C++ states access rules in each class. `Driver` keeps its rating and its `assignedRides_` list private, so other code can only add a ride with `addRide()`, count its rides, total its earnings, or get a copy of the list from `rides()`. `updateRating()` refuses values outside 1.0 to 5.0, and the `Ride` constructor throws an exception for a distance of zero or less. `Ride` keeps its own fields private too, so even subclasses store a fare through the protected method `recordFare()` (Stroustrup, 2013). Smalltalk has no access keywords: an object's instance variables can be used only by its own methods, and any method can be called by anyone (Goldberg & Robson, 1983). `Driver` has no method that returns `assignedRides`, so the list cannot be reached from outside, and `rides` returns a copy. Section 6 of each run shows both checks working (Figures A11 and A13).

Inheritance

In C++ the parent is named in the class header, as in `class SharedRide : public StandardRide`. Constructors are not inherited unless asked for, so each subclass adds a declaration such as `using Ride::Ride;` (Stroustrup, 2013). In Smalltalk a class is made by

sending `subclass:` to its parent, as in `Ride subclass: StandardRide`. Subclasses use inherited instance variables directly, so `StandardRide`'s `fare` reads `distance` with no getter, and class methods are inherited as well, so one constructor, `id:from:to:miles:`, builds every kind of ride. Both versions reuse the parent's formula the same way: `SharedRide` calls `StandardRide::fare()` in C++ and `super fare` in Smalltalk, then takes 30% off.

Polymorphism

In C++, `fare()`, `rideType()`, `needsTopDriver()`, and `rideDetails()` are `virtual` in `Ride` and marked `override` in the subclasses, and the destructor is virtual so rides are deleted correctly through `Ride` pointers. All rides sit in one `std::vector<std::shared_ptr<Ride>>`, and the loop in `main.cpp` calls `rideDetails()` and `fare()` on each, so each ride runs its own version (Figure A5). A virtual call inside a constructor runs the base version (Stroustrup, 2013), so the fare is worked out by `fare()` when needed rather than in the constructor. In Smalltalk every message is looked up in the receiver's class when it is sent (Goldberg & Robson, 1983), so no keyword is needed: the rides are kept in an `OrderedCollection`, and `rides do: [:ride | ride rideDetails]` works the same way (Figure A9). The dispatcher also asks each ride `needsTopDriver` instead of checking its class. In short, C++ makes the programmer declare access and dynamic binding and checks them when compiling, while Smalltalk applies both to every object by default.

References

Goldberg, A., & Robson, D. (1983). *Smalltalk-80: The language and its implementation*. Addison-Wesley.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

Appendix

Screenshots of the Code and Sample Output

cpp/ride.h lines 1-34

```cpp
#pragma once
// Ride and its subclasses. Fares are kept in whole cents.

#include <string>

#include "format.h"

// Base class: every ride has an ID, a route, a distance and a fare.
class Ride {
public:
    Ride(std::string rideID, std::string pickupLocation, std::string dropoffLocation,
         double distance);
    virtual ~Ride() = default;  // rides are deleted through Ride pointers

    virtual Money fare();                  // basic rate: $1.50 per mile
    virtual std::string rideType() const;  // "Basic"
    virtual bool needsTopDriver() const;   // only premium rides say yes
    virtual std::string rideDetails();     // one line: ID, type, route, miles, fare

    const std::string& rideID() const { return rideID_; }
    double distance() const { return distance_; }
    std::string route() const { return pickupLocation_ + " -> " + dropoffLocation_; }

protected:
    // Subclasses cannot touch fare_ directly; they store a fare through this.
    Money recordFare(Money cents);

private:
    std::string rideID_;
    std::string pickupLocation_;
    std::string dropoffLocation_;
    double distance_;
    Money fare_ = 0;  // the fare from the last fare() call
};
```

Figure A1

C++ Ride Base Class (cpp/ride.h)

cpp/ride.h lines 36-64

```cpp
// $2.50 booking fee + $1.80 per mile.
class StandardRide : public Ride {
public:
    using Ride::Ride;  // constructors are not inherited unless asked for
    Money fare() override;
    std::string rideType() const override;
};

// $5.00 + $3.20 per mile, never less than $15.00, top-rated drivers only.
class PremiumRide : public Ride {
public:
    using Ride::Ride;
    Money fare() override;
    std::string rideType() const override;
    bool needsTopDriver() const override;
    std::string rideDetails() override;

private:
    Money meteredFare() const;  // the fare before the $15.00 minimum
};

// A Standard ride shared with other riders: the Standard fare with 30% off.
class SharedRide : public StandardRide {
public:
    using StandardRide::StandardRide;
    Money fare() override;
    std::string rideType() const override;
    std::string rideDetails() override;
};
```

Figure A2

C++ StandardRide, PremiumRide, and SharedRide Subclasses (cpp/ride.h)

cpp/ride.cpp lines 22-68

```cpp
}

Money Ride::fare() { return recordFare(std::llround(150 * distance_)); }

std::string Ride::rideType() const { return "Basic"; }

bool Ride::needsTopDriver() const { return false; }

std::string Ride::rideDetails() {
    // rideType() and fare() are virtual, so this one method prints every kind of ride.
    return text::padRight(rideID_, 6) + text::padRight(rideType(), 10) +
           text::padRight(route(), 28) + text::padLeft(text::tenths(distance_), 5) + " mi" +
           text::padLeft(text::money(fare()), 10);
}

// ---- StandardRide -------------------------------------------------------

Money StandardRide::fare() { return recordFare(250 + std::llround(180 * distance())); }

std::string StandardRide::rideType() const { return "Standard"; }

// ---- PremiumRide --------------------------------------------------------

Money PremiumRide::meteredFare() const { return 500 + std::llround(320 * distance()); }

Money PremiumRide::fare() { return recordFare(std::max<Money>(meteredFare(), 1500)); }

std::string PremiumRide::rideType() const { return "Premium"; }

bool PremiumRide::needsTopDriver() const { return true; }

std::string PremiumRide::rideDetails() {
    std::string line = Ride::rideDetails() + "  top-rated driver";
    if (meteredFare() < 1500) line += ", minimum fare";
    return line;
}

// ---- SharedRide ---------------------------------------------------------

Money SharedRide::fare() {
    Money full = StandardRide::fare();      // reuse the parent's formula
    return recordFare((full * 7 + 5) / 10);  // 70%, rounded to the nearest cent
}

std::string SharedRide::rideType() const { return "Shared"; }

std::string SharedRide::rideDetails() { return StandardRide::rideDetails() + "  30% off Standard"; }
```

Figure A3

C++ Overridden fare() and rideDetails() Methods (cpp/ride.cpp)

cpp/driver.h lines 1-29

```cpp
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "ride.h"

class Driver {
public:
    Driver(std::string driverID, std::string name, double rating);

    void addRide(std::shared_ptr<Ride> ride);
    std::string getDriverInfo() const;
    bool updateRating(double newRating);  // false, and no change, outside 1.0 to 5.0

    const std::string& driverID() const { return driverID_; }
    const std::string& name() const { return name_; }
    double rating() const { return rating_; }
    std::size_t rideCount() const { return assignedRides_.size(); }
    Money earnings() const;
    std::vector<std::shared_ptr<Ride>> rides() const;  // a copy of the list

private:
    std::string driverID_;
    std::string name_;
    double rating_;
    std::vector<std::shared_ptr<Ride>> assignedRides_;  // only reachable through the methods above
};
```

Figure A4

C++ Driver Class With a Private Ride List (cpp/driver.h)

cpp/main.cpp lines 14-43

```cpp
int main() {
    std::cout << "Ride Sharing System (C++)\n" << std::string(64, '=') << "\n";

    RideSystem rideSystem;
    auto carlos = rideSystem.addDriver(std::make_shared<Driver>("D1", "Carlos", 4.9));
    auto aisha = rideSystem.addDriver(std::make_shared<Driver>("D2", "Aisha", 4.8));
    auto ben = rideSystem.addDriver(std::make_shared<Driver>("D3", "Ben", 4.5));
    Rider maya("R1", "Maya"), leo("R2", "Leo"), priya("R3", "Priya");

    section("1. Riders request rides");
    rideSystem.request(maya, std::make_shared<StandardRide>("S101", "Back Bay", "Logan Airport", 4.2));
    rideSystem.request(leo, std::make_shared<PremiumRide>("P102", "Cambridge", "Seaport", 5.5));
    rideSystem.request(priya, std::make_shared<SharedRide>("H103", "Fenway", "Harvard Square", 3.1));
    rideSystem.request(leo, std::make_shared<StandardRide>("S104", "Logan Airport", "Quincy", 9.8));
    rideSystem.request(maya, std::make_shared<PremiumRide>("P105", "North End", "South Boston", 1.6));
    rideSystem.request(priya, std::make_shared<SharedRide>("H106", "Somerville", "MIT", 2.5));
    rideSystem.request(priya, std::make_shared<Ride>("B107", "Brookline", "Back Bay", 3.0));

    section("2. Dispatch (premium rides need a driver rated 4.8 or higher)");
    rideSystem.dispatchAll();

    section("3. All rides in one list: rideDetails() and fare() on each, whatever its type");
    std::cout << "  " << text::padRight("ID", 6) << text::padRight("Type", 10)
              << text::padRight("Route", 28) << "Distance" << text::padLeft("Fare", 10) << "\n";
    Money total = 0;
    for (const auto& ride : rideSystem.rides()) {  // std::shared_ptr<Ride>, any subclass
        std::cout << "  " << ride->rideDetails() << "\n";
        total += ride->fare();
    }
    std::cout << "  All fares: " << text::money(total) << "\n";
```

Figure A5

C++ Demo: Mixed Ride Types in One List (cpp/main.cpp)

smalltalk/RideSharing.st lines 32-73

```
Object subclass: Ride [
    "Instance variables can only be read or changed by this object's own
     methods, including methods inherited by subclasses."
    | rideID pickupLocation dropoffLocation distance fare |
    <comment: 'Base class: every ride has an ID, a route, a distance and a fare.'>

    Ride class >> id: anID from: pickup to: dropoff miles: miles [
        "Class methods are inherited too, so every subclass gets this constructor
         and 'self new' makes an instance of whichever class received it."
        miles > 0 ifFalse: [^self error: 'distance must be greater than 0'].
        ^self new setID: anID from: pickup to: dropoff miles: miles
    ]

    setID: anID from: pickup to: dropoff miles: miles [
        <category: 'private'>
        rideID := anID.
        pickupLocation := pickup.
        dropoffLocation := dropoff.
        distance := miles
    ]

    rideID [ ^rideID ]
    distance [ ^distance ]
    route [ ^pickupLocation, ' -> ', dropoffLocation ]

    fare [
        "Basic rate: $1.50 per mile."
        ^fare := (150 * distance) rounded
    ]

    rideType [ ^'Basic' ]

    needsTopDriver [ ^false ]

    rideDetails [
        "rideType and fare are looked up on the receiver at run time, so this
         one method prints every kind of ride."
        ^(Fmt pad: rideID to: 6), (Fmt pad: self rideType to: 10),
            (Fmt pad: self route to: 28), (Fmt padLeft: (Fmt tenths: distance) to: 5), ' mi',
            (Fmt padLeft: (Fmt money: self fare) to: 10)
    ]
]
```

Figure A6

Smalltalk Ride Base Class (smalltalk/RideSharing.st)

smalltalk/RideSharing.st lines 75-114

```
75  Ride subclass: StandardRide [
76      <comment: '$2.50 booking fee + $1.80 per mile.'>
77
78      fare [ ^fare := 250 + (180 * distance) rounded ]
79
80      rideType [ ^'Standard' ]
81  ]
82
83  Ride subclass: PremiumRide [
84      <comment: '$5.00 + $3.20 per mile, never less than $15.00, top-rated drivers only.'>
85
86      meteredFare [
87          "The fare before the $15.00 minimum."
88          ^500 + (320 * distance) rounded
89      ]
90
91      fare [ ^fare := self meteredFare max: 1500 ]
92
93      rideType [ ^'Premium' ]
94
95      needsTopDriver [ ^true ]
96
97      rideDetails [
98          ^super rideDetails, '  top-rated driver',
99              (self meteredFare < 1500 ifTrue: [', minimum fare'] ifFalse: [''])
100     ]
101 ]
102
103 StandardRide subclass: SharedRide [
104     <comment: 'A Standard ride shared with other riders: the Standard fare with 30% off.'>
105
106     fare [
107         "super fare reuses the parent's formula; 70%, rounded to the nearest cent."
108         ^fare := (super fare * 7 / 10) rounded
109     ]
110
111     rideType [ ^'Shared' ]
112
113     rideDetails [ ^super rideDetails, '  30% off Standard' ]
114 ]
```

Figure A7

Smalltalk StandardRide, PremiumRide, and SharedRide Subclasses

smalltalk/RideSharing.st lines 116-165

```
Object subclass: Driver [
    | driverID name rating assignedRides |
    <comment: 'assignedRides has no accessor: it is reached only through addRide:, rideCount, earnings and rides.'>

    Driver class >> id: anID name: aName rating: aNumber [
        ^self new setID: anID name: aName rating: aNumber
    ]

    setID: anID name: aName rating: aNumber [
        <category: 'private'>
        driverID := anID.
        name := aName.
        rating := aNumber.
        assignedRides := OrderedCollection new
    ]

    driverID [ ^driverID ]
    name [ ^name ]
    rating [ ^rating ]

    addRide: aRide [
        assignedRides add: aRide.
        ^aRide
    ]

    updateRating: aNumber [
        "Answers false, and changes nothing, outside 1.0 to 5.0."
        (aNumber between: 1.0 and: 5.0) ifFalse: [^false].
        rating := aNumber.
        ^true
    ]

    rideCount [ ^assignedRides size ]

    earnings [ ^assignedRides inject: 0 into: [:sum :ride | sum + ride fare] ]

    rides [
        "A copy, so callers cannot change the real list."
        ^assignedRides copy
    ]

    getDriverInfo [
        | ids |
        ids := WriteStream on: String new.
        assignedRides do: [:ride | ids nextPutAll: ride rideID] separatedBy: [ids nextPutAll: ', '].
        ^(Fmt pad: driverID to: 4), (Fmt pad: name to: 8), 'rating ', (Fmt tenths: rating),
            '   ', self rideCount printString, ' rides   earned ', (Fmt money: self earnings),
            (String with: Character nl), '      rides: ', ids contents
    ]
]
```

Figure A8

Smalltalk Driver Class (smalltalk/RideSharing.st)

smalltalk/main.st lines 1-36

```
"Ride Sharing System, GNU Smalltalk version: the demo.
 Run from this folder:  gst RideSharing.st main.st"

| rideSystem carlos aisha ben maya leo priya total result copy |

Transcript showCr: 'Ride Sharing System (GNU Smalltalk)'; showCr: (String new: 64 withAll: $=).

rideSystem := RideSystem new.
carlos := rideSystem addDriver: (Driver id: 'D1' name: 'Carlos' rating: 4.9).
aisha := rideSystem addDriver: (Driver id: 'D2' name: 'Aisha' rating: 4.8).
ben := rideSystem addDriver: (Driver id: 'D3' name: 'Ben' rating: 4.5).
maya := Rider id: 'R1' name: 'Maya'.
leo := Rider id: 'R2' name: 'Leo'.
priya := Rider id: 'R3' name: 'Priya'.

Transcript cr; showCr: '1. Riders request rides'.
rideSystem rider: maya requests: (StandardRide id: 'S101' from: 'Back Bay' to: 'Logan Airport' miles: 4.2).
rideSystem rider: leo requests: (PremiumRide id: 'P102' from: 'Cambridge' to: 'Seaport' miles: 5.5).
rideSystem rider: priya requests: (SharedRide id: 'H103' from: 'Fenway' to: 'Harvard Square' miles: 3.1).
rideSystem rider: leo requests: (StandardRide id: 'S104' from: 'Logan Airport' to: 'Quincy' miles: 9.8).
rideSystem rider: maya requests: (PremiumRide id: 'P105' from: 'North End' to: 'South Boston' miles: 1.6).
rideSystem rider: priya requests: (SharedRide id: 'H106' from: 'Somerville' to: 'MIT' miles: 2.5).
rideSystem rider: priya requests: (Ride id: 'B107' from: 'Brookline' to: 'Back Bay' miles: 3.0).

Transcript cr; showCr: '2. Dispatch (premium rides need a driver rated 4.8 or higher)'.
rideSystem dispatchAll.

Transcript cr; showCr: '3. All rides in one list: rideDetails() and fare() on each, whatever its type'.
Transcript showCr: '  ', (Fmt pad: 'ID' to: 6), (Fmt pad: 'Type' to: 10), (Fmt pad: 'Route' to: 28),
    'Distance', (Fmt padLeft: 'Fare' to: 10).
total := 0.
rideSystem rides do: [:ride |
    "Any object that understands rideDetails and fare can be in this list."
    Transcript showCr: '  ', ride rideDetails.
    total := total + ride fare].
Transcript showCr: '  All fares: ', (Fmt money: total).
```

Figure A9

Smalltalk Demo: Mixed Ride Types in One List (smalltalk/main.st)

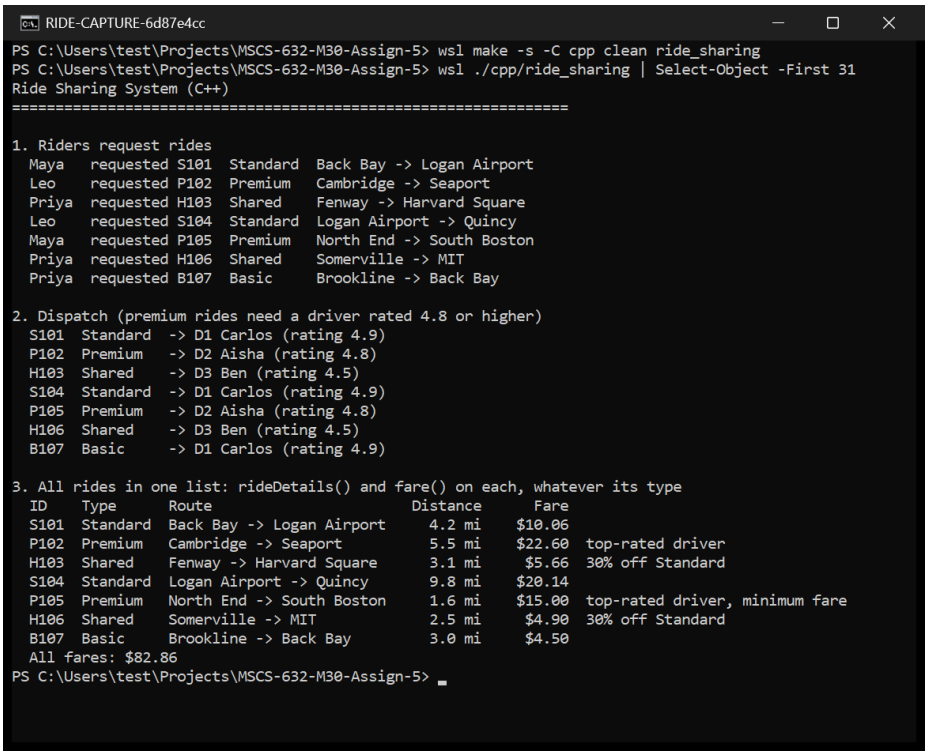


Figure A10

C++ Version Built and Run, Part 1: Requests, Dispatch, and All Rides

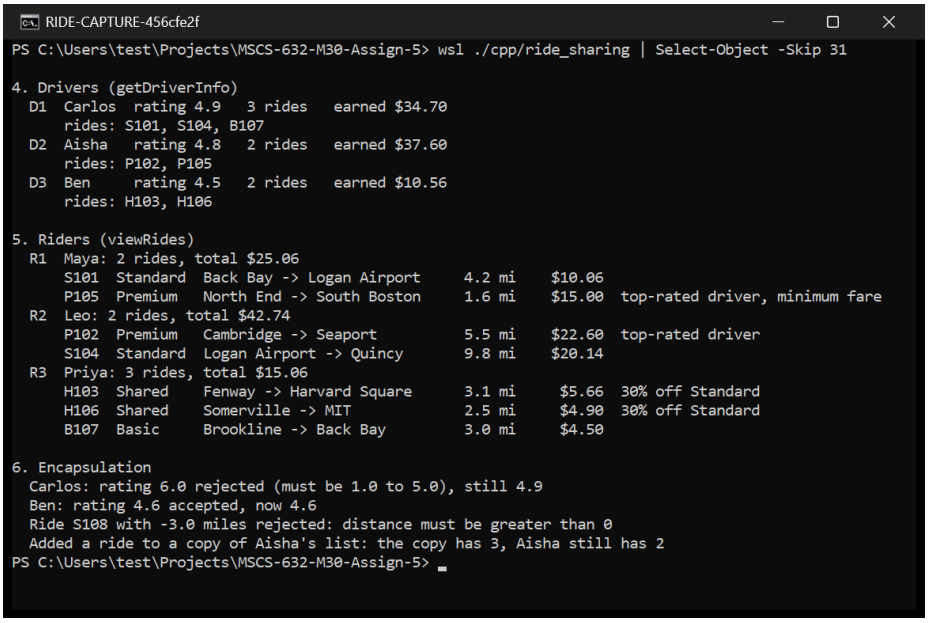


Figure A11

C++ Version, Part 2: Drivers, Riders, and Encapsulation Checks

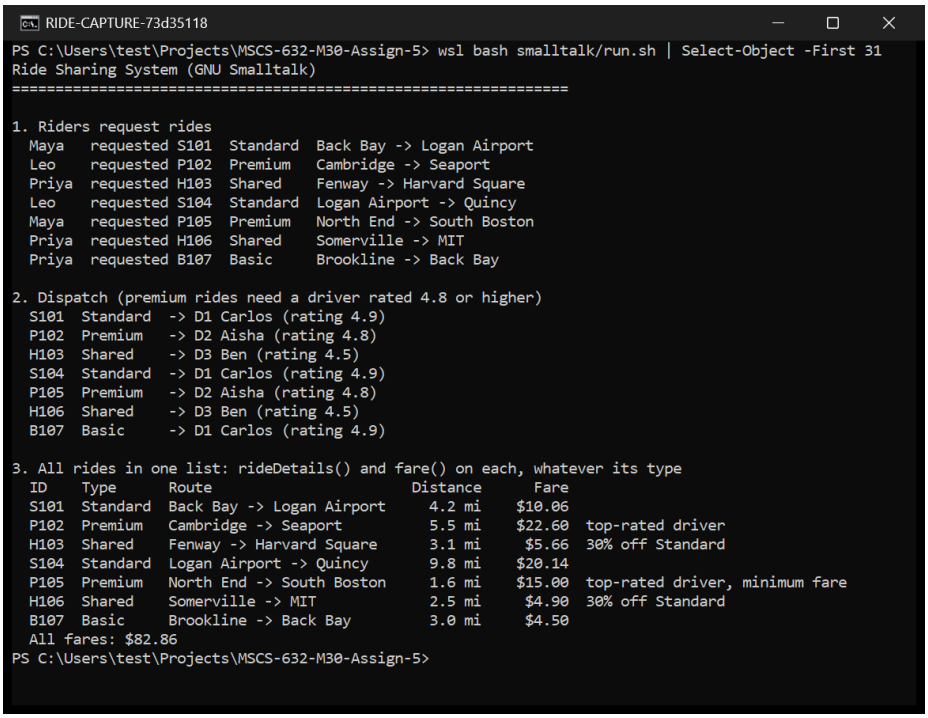


Figure A12

Smalltalk Version Run, Part 1: Requests, Dispatch, and All Rides

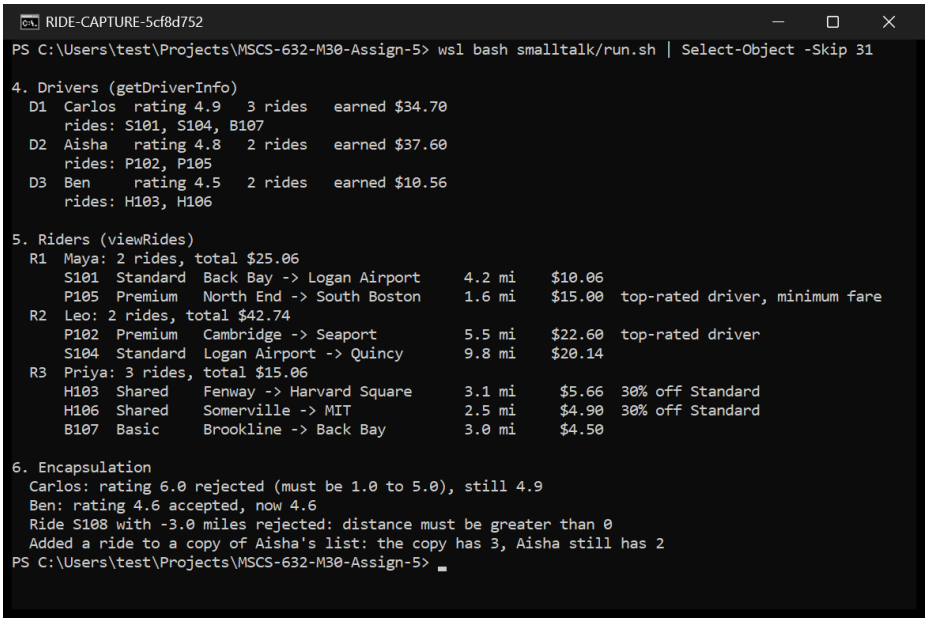


Figure A13

Smalltalk Version, Part 2: Drivers, Riders, and Encapsulation Checks

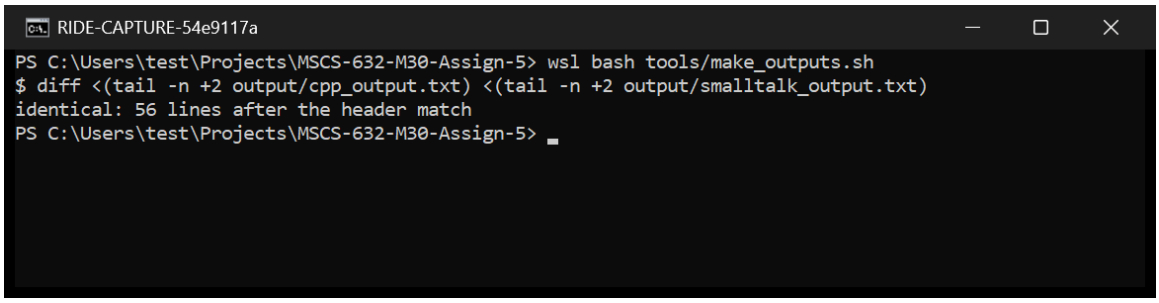


Figure A14

Both Outputs Compared: Identical After the Header Line